

Descripción del Pipeline de Datos

Entregable 4 · Práctica de Arquitectura y Diseño de Sistemas 2026

Grupo 17 · Marketplace de Fragancias — *Fragancio Elegancio*

PIPELINES DOCUMENTADOS

- 1. ETL de Catálogo de Fragancias Batch / ETL
- 2. Procesamiento de Compra Event-Driven
- 3. Motor de Recomendaciones On-Demand

PIPELINE 1

ETL de Catálogo de Fragancias

Batch · ETL

Carga y normaliza el catálogo inicial de perfumes desde una API externa hacia la base de datos del sistema, permitiendo que el marketplace tenga productos desde el primer arranque.

▼ PASO 1 — ORIGEN DE LOS DATOS

¿Qué datos?	Perfumes en formato JSON: name, brand, notes_top[], notes_middle[], notes_base[], description, gender, image_url.
¿Desde dónde?	API externa: <code>https://perfumapidatabase.onrender.com/perfumes?limit=100</code>
¿Con qué frecuencia llegan?	Una única petición HTTP GET al momento de ejecutar el seed. No hay ingestión continua.
¿Volumen estimado?	Hasta 100 perfumes por ejecución (~15 KB de JSON). Operación de setup, no recurrente.

▼ PASO 2 — DISPARADOR / TRIGGER

¿Cuándo se ejecuta?	Manualmente por el equipo de desarrollo durante la inicialización del sistema, mediante el comando <code>npx prisma db seed</code> .
¿Quién lo dispara?	El administrador del sistema (desarrollador) desde la terminal. No existe scheduler ni trigger automático.
▼ PASO 3 — PROCESAMIENTO	
¿Qué componente lo ejecuta?	Script <code>prisma/seed.ts</code> — Node.js con Prisma Client. Se ejecuta en el contexto del servidor de base de datos.
Paso 1 — Extracción	Fetch HTTP GET a la API externa. Se acepta respuesta como array directo o dentro del campo <code>perfumes</code> (manejo de dos formatos posibles).
Paso 2 — Validación	Se descartan registros sin <code>name</code> o <code>brand</code> (campos obligatorios para unicidad en la BD). El resto continúa el flujo.
Paso 3 — Normalización	Transformaciones aplicadas: <code>name</code> y <code>brand</code> → <code>.trim()</code> <code>notes_top[]</code>, <code>notes_middle[]</code>, <code>notes_base[]</code> → <code>.join(", ")</code> para almacenarlas como texto plano (formato compatible con el motor de recomendaciones) - Campos opcionales (<code>description</code>, <code>image_url</code>, <code>gender</code>) → <code>null</code> si ausentes
Paso 4 — Enriquecimiento	Se asignan valores por defecto para la primera variante de cada producto: <code>stock = 10</code> , <code>volumen = 50 ml</code> , <code>precio = 50 000</code> , <code>ranking = 1</code> , <code>concentracion = gender</code> (del perfume original).
Paso 5 — Carga (Upsert)	Se ejecuta <code>prisma.producto.upsert</code> con clave única (<code>nombre</code> , <code>marca</code>). Si el producto ya existe, se actualizan sus campos. Solo si no tiene ninguna variante asociada se crea una con los valores por defecto.
¿Hay lógica de error?	Si la API externa falla (<code>HTTP != 2xx</code>), se lanza una excepción que detiene la ejecución y devuelve exit code 1. Registros individuales inválidos (sin nombre/marca) se descartan silenciosamente con <code>continue</code> . No hay reintentos automáticos.
▼ PASO 4 — SALIDA / DESTINO	
¿Qué se produce?	Registros persistidos en las tablas <code>Producto</code> (<code>nombre</code> , <code>marca</code> , <code>stock</code> , <code>notas olfativas</code> , <code>imagen</code>) y

	VarianteProducto (precio, volumen, concentración, ranking).
¿Dónde se guarda?	Base de datos PostgreSQL gestionada por Prisma ORM. Tabla Producto con unicidad compuesta (nombre, marca); tabla VarianteProducto con FK id_producto.
¿Quién consume el resultado?	El catálogo del marketplace: GET /api/catalogo, la página /catalogo y el motor de recomendaciones. El resultado es el punto de partida de todos los demás pipelines.
▼ PASO 5 — DECISIÓN ARQUITECTÓNICA	
¿Por qué este enfoque?	Se eligió un ETL batch manual porque el catálogo de fragancias es estático (no cambia en tiempo real): un marketplace de este tipo parte de un inventario inicial fijo que los vendedores luego amplían. El batch evita la complejidad de un pipeline streaming para datos que no lo requieren.
Trade-offs aceptados	• No hay actualización automática del catálogo: si la API externa añade nuevos perfumes, hay que re-ejecutar el seed manualmente. • Sin reintentos automáticos: si la API externa cae durante el seed, hay que repetir el proceso completo. • Precios y stock son placeholders: los vendedores deben ajustar estos valores vía la API de inventario tras la carga inicial.

PIPELINE 2

Procesamiento de Compra

Event-Driven

Orquesta el flujo completo de una compra: desde que el comprador confirma su carrito hasta que el pago es procesado por una pasarela externa y se genera la factura y el envío. Se compone de dos eventos encadenados: el checkout iniciado por el usuario y el webhook de confirmación de la pasarela de pago.

▼ PASO 1 — ORIGEN DE LOS DATOS

¿Qué datos?

Evento A (Checkout): Token de sesión Clerk del comprador + estado del Carrito activo: id_carrito, lista de {id_producto, cantidad}.

	Evento B (Webhook): Payload JSON de la pasarela: {id_carrito, estado} donde estado ∈ {aprobado, rechazado}. Incluye header x-webhook-signature (HMAC-SHA256).
¿Desde dónde?	Evento A: HTTP POST del browser del comprador a /api/checkout. El carrito se lee desde PostgreSQL. Evento B: HTTP POST de la pasarela externa (Stripe / MercadoPago) a /api/pagos/webhook.
¿Con qué frecuencia llegan?	Ante cada intento de compra de un usuario. El webhook llega minutos después del checkout, una vez que la pasarela confirma el pago.
¿Volumen estimado?	Bajo en un marketplace académico: estimado < 100 compras/día. Cada transacción involucra ~5 ítems en promedio. El impacto es acotado pero la criticidad es alta (dinero real).
▼ PASO 2 — DISPARADOR / TRIGGER	
¿Cuándo se ejecuta?	Evento A: Acción del usuario (comprador hace clic en "Confirmar compra"). Evento B: Callback asíncrono de la pasarela de pago, típicamente segundos o minutos después del Evento A.
¿Quién lo dispara?	Evento A: El browser del comprador vía HTTP POST. Evento B: El sistema externo de pagos (Stripe / MercadoPago) vía webhook HTTP POST.
▼ PASO 3 — PROCESAMIENTO	
¿Qué componente lo ejecuta?	Evento A: src/app/api/checkout/route.ts + src/lib/stock.ts — Next.js API Route (Node.js). Evento B: src/app/api/pagos/webhook/route.ts — Next.js API Route (Node.js).
— EVENTO A: Checkout —	
Paso 1 — Autenticación y autorización	Se verifica el token Clerk. Si el usuario no está autenticado → 401. Si el rol no es comprador → 403. Se resuelve el id_usuario del Comprador en la BD vía su email.
Paso 2 — Recuperación del Carrito	Se busca el Carrito con estado = "activo" del comprador. Si no existe → 404. Si está vacío → 400.

Paso 3 — Transacción atómica (stock.ts)

Dentro de `prisma.$transaction`, por cada ítem:

1. Consulta stock actual del Producto.
2. Valida que `stock ≥ cantidad`. Si falla, toda la transacción se revierte.
3. Decrementa stock: `UPDATE Producto SET stock = stock - cantidad`.
4. Acumula `precio × cantidad` para calcular el total.

Al finalizar el loop: crea registro Pago con `estado = "pendiente"` y actualiza `Carrito.estado = "convertido"`.

— EVENTO B: Webhook de Pago —

Paso 4 — Verificación de firma

Se recalcula la firma HMAC-SHA256 del body con el `WEBHOOK_SECRET` y se compara con el header `x-webhook-signature` usando `crypto.timingSafeEqual` (resistente a timing attacks). Firma inválida → 401.

Paso 5 — Validación e idempotencia

Se verifica que exista un Pago asociado al `id_carrito`. Si el pago ya fue aprobado previamente → 409 (idempotencia: protege contra webhooks duplicados de la pasarela).

Paso 6 — Actualización de estado y generación de comprobantes

Si `estado = "aprobado"`:

1. Actualiza `Pago.estado = "aprobado"`.
2. Recalcula el importe total desde `CarritoProducto × precios de VarianteProducto`.
3. Crea registro Factura con `importe_total` y fecha de emisión (CUID autogenerado).
4. Crea registro Envío con `estado = "preparando"`.

Si `estado = "rechazado"`: solo actualiza `Pago.estado = "rechazado"`. No se crea Factura ni Envío.

¿Hay lógica de error?

La transacción atómica del Evento A garantiza que o todo el stock se decrementa y el Pago se crea, o nada cambia. Si falla → 409 `CHECKOUT_FALLIDO` con mensaje descriptivo. El Evento B es idempotente: puede recibirse múltiples veces sin efectos adicionales una vez aprobado.

▼ PASO 4 — SALIDA / DESTINO

¿Qué se produce?

Evento A: Registro Pago con `estado = "pendiente"`. Respuesta HTTP 201 con `{id_pago, importe_total, estado}`.

Evento B: Pago actualizado + registro Factura (con

	nro_factura) + registro Envío con estado = "preparando".
¿Dónde se guarda?	PostgreSQL: tablas Pago (id_carrito único), Factura (id_pago único, nro_factura CUID), Envío (id_carrito único).
¿Quién consume el resultado?	El frontend del comprador consulta el resultado vía GET /api/pedidos/{id_carrito}. El vendedor consulta el estado del pedido y actualiza el envío vía PATCH /api/envios/{id_carrito}. El sistema de notificaciones (Resend / Nodemailer) envía el email de confirmación de forma asíncrona.
▼ PASO 5 — DECISIÓN ARQUITECTÓNICA	
¿Por qué este enfoque?	Se eligió event-driven desacoplado (checkout → pasarela → webhook) para no bloquear al usuario esperando la respuesta del sistema de pagos externo. El checkout responde inmediatamente con el id_pago y la pasarela confirma de forma asíncrona. Esto mejora la experiencia de usuario y desacopla el sistema de las latencias externas. La transacción <code>\$transaction</code> de Prisma garantiza atomicidad en el decremento de stock para prevenir sobreventa bajo concurrencia.
Trade-offs aceptados	• El stock se decrementa antes de confirmar el pago: si el pago es rechazado, el stock no se restaura automáticamente. Esto se acepta para simplificar el flujo en el contexto del proyecto. • Sin reintentos en el webhook: si el servidor cae mientras procesa el webhook, la pasarela puede reintentar según su configuración. El diseño idempotente lo maneja correctamente. • Sin reserva temporal expirada: el diseño original preveía liberar el stock luego de 5 minutos si el pago no se completaba; esta funcionalidad quedó simplificada para el alcance actual.

PIPELINE 3

Motor de Recomendaciones

On-Demand (Request-Response)

Calcula en tiempo real los productos más similares a uno dado, usando similitud de

Jaccard sobre las notas olfativas (salida, corazón y fondo) e ingredientes. El resultado se devuelve al usuario sin persistencia: cada consulta recalcula desde cero.

▼ PASO 1 — ORIGEN DE LOS DATOS

¿Qué datos?	Campos textuales de la tabla Producto: notas_salida, notas_corazon, notas_fondo e ingrediente (todos en formato texto libre, separados por comas). También se recupera la variante de mayor ranking (precio, concentracion) para incluirla en la respuesta.
¿Desde dónde?	Base de datos PostgreSQL vía Prisma. Dos consultas: una para el producto base y una para todos los productos candidatos con stock > 0.
¿Con qué frecuencia llegan?	Ante cada visita de un usuario a la página de detalle de un producto (GET /api/recomendaciones?productoId=N).
¿Volumen estimado?	El catálogo tiene ~100 productos (cargados por el seed). Cada consulta compara el producto base contra los ~99 restantes. La operación es completamente en memoria, con complejidad O(n) sobre el catálogo.

▼ PASO 2 — DISPARADOR / TRIGGER

¿Cuándo se ejecuta?	En tiempo real, ante cada request HTTP GET al endpoint /api/recomendaciones. No hay programación periódica ni batch.
¿Quién lo dispara?	El componente del frontend que muestra la sección "Productos similares" en la página de detalle de un perfume. El parámetro productId identifica el producto base.

▼ PASO 3 — PROCESAMIENTO

¿Qué componente lo ejecuta?	src/lib/recomendaciones.ts (lógica) invocado por src/app/api/recomendaciones/route.ts (API Route). Node.js, procesamiento completamente en memoria.
Paso 1 — Obtención del producto base	Query a la BD para recuperar notas_salida, notas_corazon, notas_fondo e ingrediente del producto con el id_producto recibido. Si no existe → 404.
Paso 2 — Obtención de candidatos	Query a la BD para recuperar todos los productos con stock > 0, excluyendo el producto base. Se incluye la variante de mayor ranking para obtener precio y concentración.

**Paso 3 —
Tokenización**

Función `tokenize(text)`: convierte cada campo de notas a un `Set<string>` de tokens en minúsculas, dividiendo por comas y espacios. Ejemplo: "Rose, Jasmine, Iris" → {"rose", "jasmine", "iris"}.

**Paso 4 — Cálculo de
similitud (Jaccard)**

Función `jaccardSimilarity(A, B) = |A ∩ B| / |A ∪ B|`. Se calcula para cada par (producto base, candidato) con pesos ponderados:

Campo	Peso	Justificación
notas_corazon	40 %	Corazón del perfume, define su identidad
notas_salida	30 %	Primera impresión olfativa
notas_fondo	20 %	Persistencia y drydown
ingrediente	10 %	Composición química secundaria

**Paso 5 — Ranking y
selección**

Los candidatos se ordenan por `score` descendente. Se devuelven los primeros `limit` resultados (default 6, máximo 20). El campo `score` se elimina de la respuesta final.

¿Hay lógica de error?

Validación del parámetro `productoId` en la API Route (entero positivo). Si el producto no existe → 404. No hay lógica de caché ni reintentos; los errores de BD se propagan como 500.

▼ PASO 4 — SALIDA / DESTINO

¿Qué se produce?

Lista JSON de hasta 6 productos recomendados, cada uno con: `id_producto`, `nombre`, `marca`, `precio`, `concentracion`, `notas_salida`, `notas_corazon`, `notas_fondo`.

¿Dónde se guarda?

No se persiste. El resultado es efímero: existe únicamente en la respuesta HTTP. No hay caché (cada consulta recalcula en tiempo real).

¿Quién consume el resultado?

El componente de frontend de la página de detalle de producto, que renderiza la sección "Te puede interesar". El usuario final ve las recomendaciones como cards de producto.

▼ PASO 5 — DECISIÓN ARQUITECTÓNICA

¿Por qué este enfoque?

Se eligió similitud de Jaccard sobre texto de notas olfativas porque los datos disponibles son descriptivos (no hay historial

de compras ni ratings). Es un algoritmo sin estado, determinístico y que aprovecha directamente la información del catálogo cargado por el pipeline ETL. El procesamiento en memoria es viable dado que el catálogo es pequeño (~100 productos). Para el dominio de fragancias, las notas olfativas son el mejor proxy de similitud percibida por el usuario.

Trade-offs aceptados

- **Sin personalización por usuario:** las recomendaciones son iguales para todos los usuarios que consulten el mismo producto. No se considera historial de compras ni preferencias individuales.
- **Sin caché:** cada request recalcula la similitud contra todo el catálogo. Con catálogos grandes (>10 000 productos) esto se volvería un cuello de botella.
- **Calidad ligada a la riqueza de los datos:** si las notas de un perfume están incompletas o mal cargadas, las recomendaciones para ese producto serán de baja calidad.